

02 这个想法

作者 NILUSHANAN KULASINGHAM

阅读约 14 分钟 · 可交互

一个能往前推演的模型，让你在真的付出代价之前先把一个动作试一遍。麻烦在于：一个强到能找出最佳方案的搜索，也强到能找出模型出错的地方。

网球的每一分都从发球开始。一名球员把球抛起来，用尽全力击出。另一名站在网另一侧大约二十四米开外，必须在球飞过去之前把拍子碰到它。在职业水平上，球离拍时的速度大概是每小时 190 公里，所以两个人之间只隔着大约半秒钟。

半秒钟不够用。从注意到有东西、到身体开始动，光这一段就要花掉将近四分之一秒，而这还没算上整个人要跑过一段距离再挥拍。

所以厉害的那些人并不是在看球。对职业比赛的研究把「一个动作最早有可能是对球做出的反应」这条线，划在球拍击中球之后大约 140 到 160 毫秒。在一项针对高水平接发球者的研究里，横向移动大约在 177 毫秒开始，正好压在这条线上。他们做的事情，大部分在击球那一刻之前就已经定下来了。他们读的是抛球、肩膀和拍面的角度，然后朝着球还没到的那个位置移动。

这并不能证明网球运动员脑子里装着一台物理引擎。它证明的是一件更小、也更有用的事：过了某个速度以后，做得好不好，取决于还没发生的事情。

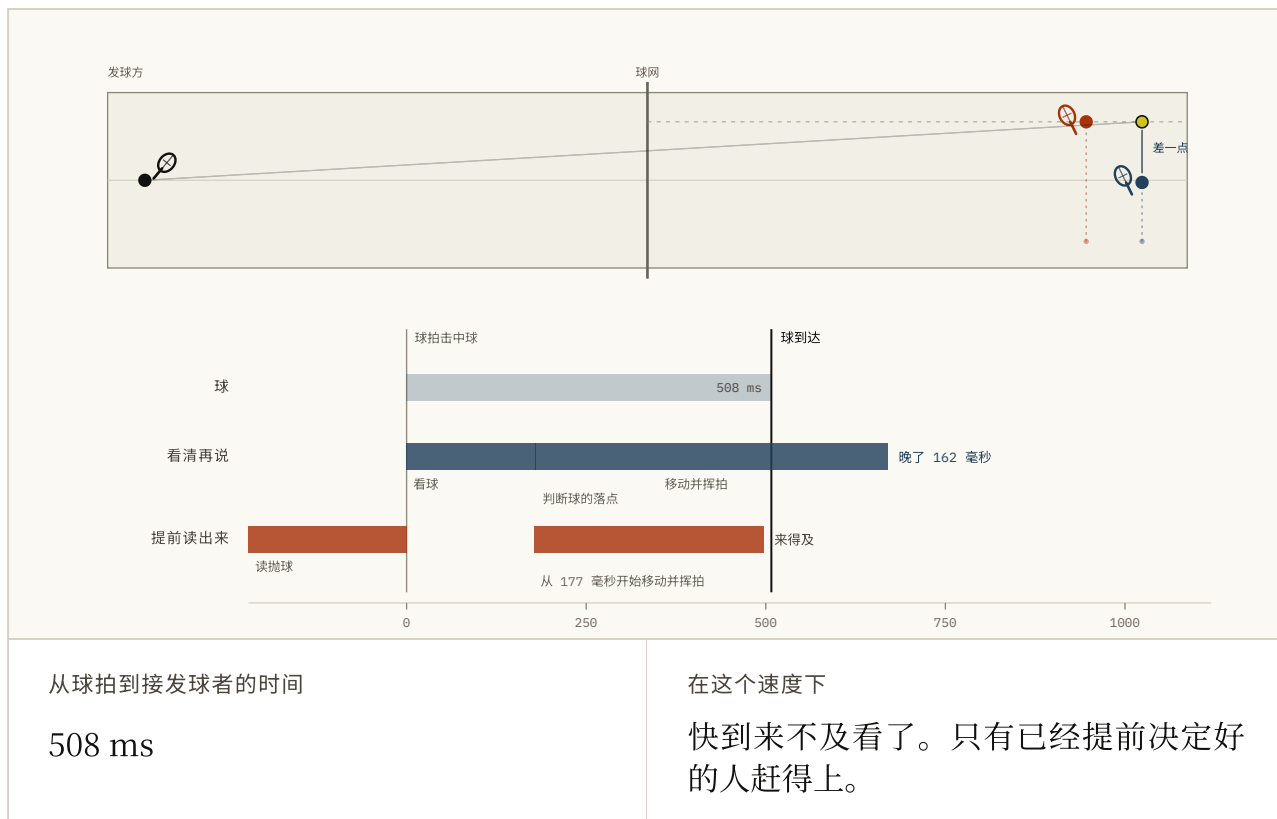


FIG. 2.1 那半秒钟都花到哪里去了。把发球速度往上拖，「球到达」这条线就会往左走，走进接发球者的时间线里。大概到一百四十五左右，那套「看清再说」的方案就塞不进去了，还能接到的只剩下在抛球阶段就已经决定的那一套。两个接发球者的时间预算是示意性的；177 毫秒是实测的。

拿上你手里有的。往前推演。朝着接下来会发生的地方移动。

这就是动力学模型。在「世界模型」这个说法今天涵盖的所有东西里，它是最老的一个，也是这个词当初被造出来时指的那个。它到底是什么，有了它能换来什么，以及它身上那个再多工程也没能去掉的失败方式。

有一件事得先说清楚，因为它会把后来去读论文的人绊倒。模型不做动作，它预测会发生什么。读这些预测并做出选择的是另一个东西，那个东西有自己的名字：**策略**（真正选出动作的那一部分，不管它是靠使劲想还是靠纯反射来选）。Ha 与 Schmidhuber 自己的系统把第三个模块叫做 controller，而那个模块并不是世界模型。这个区分在这一章里从头贯到尾。

模型里到底装了什么

它的形状并不复杂：你在哪里，你做了什么，接下来是什么。动力学模型 (dynamics 就是研究事物如何随时间变化，所以动力学模型就是一个关于「接下来会发生什么」的模型)学的就是这个，通常还会顺带学「这一步值多少」。

值得放慢一点的，是「你在哪里」这几个字被允许指什么。它不一定得是位置和速度。PlaNet 从图像里把它算出来，而且从头到尾没给其中任何一维起过名字。MuZero 走得更远，它根本不重建场景。它的模型产出的是三样说不上是什么的数值，没有画面：这一步有多好、之后会有多好、该试什么。最新的那些干脆连画面都不要了，跑起来完全没有解码器 (网络里负责把压缩过的描述还原成你能看的東西的那一半)。

$$\hat{p}(\underline{z}_{t+1}, \underline{r}_t \mid \underline{z}_t, \underline{a}_t)$$

在给定你现在在哪里和你可能采取的一个动作的条件下，该期待你会落到哪里以及这一步值多少是什么。

FIG. 2.2 整章讲的就是这个对象。把鼠标移到任意一个符号上，或者它下面的任意一个短语上。关键是那顶帽子：它表示估计出来的，而这一章后半段所有的失败，都是这顶帽子到期结账。

所以模型不必长得像世界，它只需要留住世界的行为里够用的那一部分。

所以「它够不够逼真」是问这类东西时错误的第一个问题。一个画不出象棋里的象的棋局模型，仍然可以是一个非常好的棋局模型。

一次预测本身也不是有用的那部分。有用的是你可以把答案再喂回前端。给它一串没有人执行过的动作，让它去读自己的输出，你就得到一段没有人见过的未来。这叫一次推演 (一次想象出来的向前推进：预测，把预测喂回去，再预测)，它的长度叫做视野。

这才是真正干活的那个性质，而且值得直说它不是什么。不是这东西看起来有多真，而是你能不能在没人执行过的动作下把它往前推，然后在这些动作里挑挑拣拣。

这个想法比人们通常引用的那些论文还要老。1990 年，Thrun、Möller 与 Linden 就已经把一个学出来的世界模型串起来，用它去挑未来的动作，论文标题上写的就是这几个字。

缓存下来的答案

两辆车，两堵一模一样的墙。

第一辆在墙近到某个固定距离时刹车。这个距离是某个人当年按预期的速度调好的，调完就发布了。第二辆带着一个关于自己刹车行为的模型，每一拍都在问：如果我现在刹，我会停在哪里？

在第一辆车被调校的那个速度上，两辆车的行为一模一样。

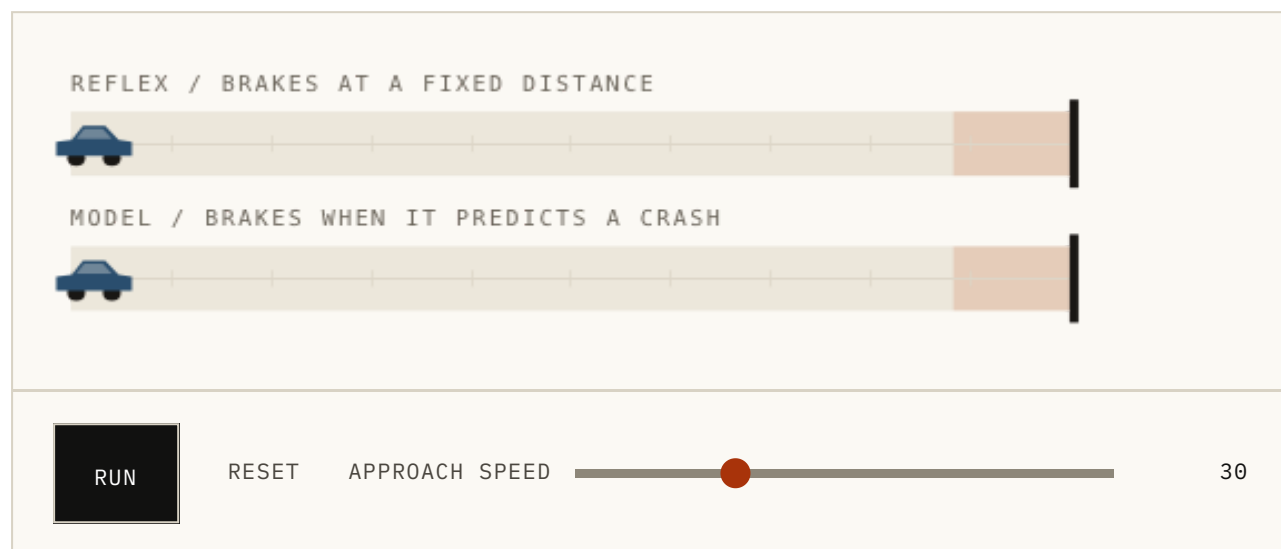


FIG. 2.3 大约在四十以下，两辆车看不出区别，而这正是这个演示里重要的那一半。把速度往上拖，看看哪一辆察觉到了。这里比较的是一条固定规则和一个会重新计算的东西，而不是笼统的「无模型」对「有模型」。

要留意这幅图到底在比什么。一个真正的、背后没有模型的策略，并不是一条固定规则。它可以是一个记得自己已经看过什么的大网络 (循环网络：它把一些东西从每一步交到下一步，所以算是有某种记忆)。它会根据看到的東西改变自己的行为。它可以比屏幕上的任何东西聪明得多。没有模型不等于没有脑子。

真正留得住的区别，是思考发生在什么时候。策略在训练阶段就把答案定下了一大半，它留下来的接近于一套答案：在这种情形下，做那件事。模型留下的是另一样东西：在这种情形下，如果你做了那件事，你会得到这个。前者是缓存。后者让你在知道了真正的问题之后，现算一个新答案出来。

反射是一个缓存下来的答案。模型是在问题变了以后，让你能算出一个新答案的那个东西。缓存更快。你只需要知道自己手里那份什么时候过期了。

缓存通常是对的，这也是为什么生物学和工程学里有那么大部分是由缓存构成的。有意思的部分从问题挪出了当初准备答案的那个范围时才开始。

来得更快一点，触发还是在同一个距离上发生。那条规则里含着一个关于「停下来要多久」的假设，而它内部没有任何东西知道自己正在做假设。模型车之所以改变了它的决定，是因为它的停车点挪了，而它看得见这一点挪动。

这些都不是白来的。模型要花计算时间，要学出来，而且可能是错的。所以真实的系统落在一条连续的谱上，而不是一条线的某一边。Dyna 把真实经验和模型编出来的经验混在一起用。Dreamer 在自己的想象里学策略。MuZero 把一个学出来的模型和搜索配在一起。

问题不是要不要带一个模型，而是哪些答案值得提前定好，哪些留到你知道自己真正面对的是什么之后再算。

有了它能换来什么

三样东西，而它们骨子里是同一类：你可以对还没发生的未来动手。

你可以在付出代价之前先试一个动作。把目标放在一个障碍物后面。一个规划器 (在真正定下来之前, 把各种动作试一遍的那一部分) 写下一串动作, 在模型里跑一遍, 给结果打个分, 然后扔掉, 再写一串。PlaNet 就是这么一边开一边做的, 在它自己对场景的压缩描述里, 起点是原始像素。

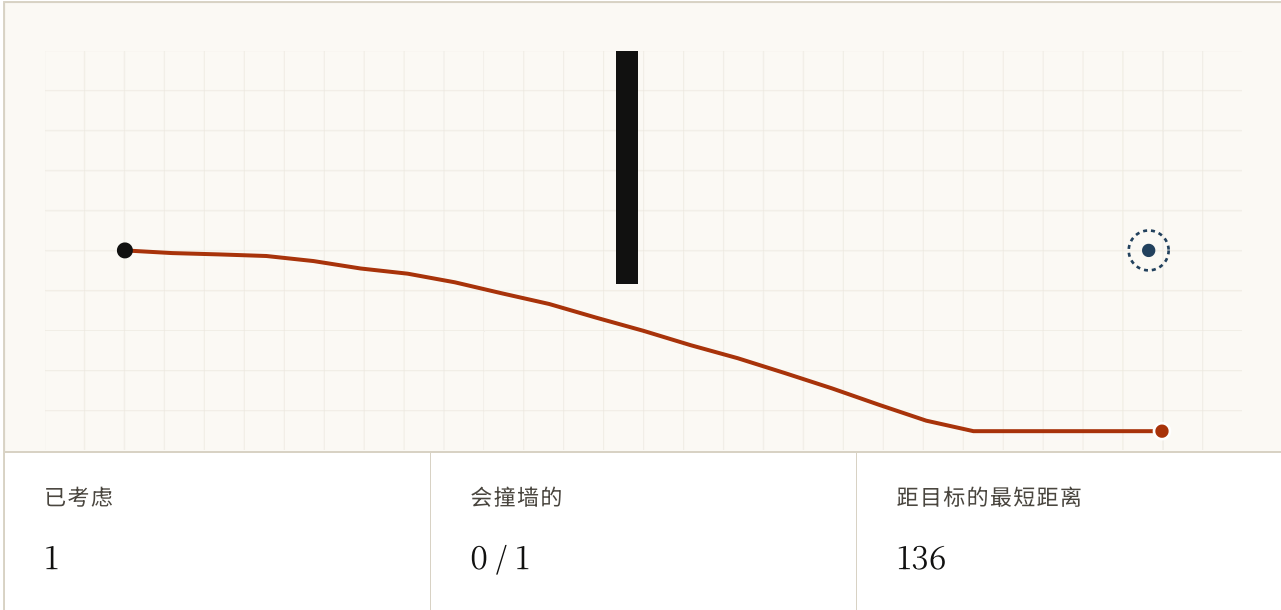


FIG. 2.4 规划器写下一串串动作, 把每一串都放进同一个模型里跑, 然后留下最好的那串。浅色的线是被否掉的。把预算调高, 方案会变好, 但只在模型还诚实地给这些候选排序的前提下成立。图 2.6 就是它不诚实之后的样子。

只有一个候选时, 它基本上是在猜。有两百个时, 它是在挑。这中间世界什么都没变。所有的提升都来自「看见」和「动手」之间的那点思考, 而模型正是让这笔交易成为可能的东西。

MuZero 展示了这条路能走多远。它从来没被训练去重画一帧游戏画面或者摆出一个棋盘, 也从来没有人告诉它国际象棋、围棋或将棋的规则。它只学了它的搜索真正会去读的那几个量, 而这三种棋它都下得很好。模型不必把将要发生的一切都复现出来, 它只需要留住你眼前这个决定所依赖的东西。

你可以在犯错很便宜的地方练习。Sutton 的 Dyna 早在 1990 年就是这么分的: 从真实经验里学一个模型, 然后让模型再编出更多经验, 并且也从这些经验里学。Dreamer 把这个把戏放到了正中间: 想象出很长的推演, 从想象出来的这些里学出行

为，等到真要动手的时候已经没什么要现算的了。它的第三个版本用同一套配方跑了 150 多个任务，没有为其中任何一个重新调参。

这里有一句很诱人的说法：*现实很贵，想象免费*。它很好记，而且在一个值得纠正的地方是错的。想象是要花计算时间的，有时候还花掉惊人的量。它省下来的是和真实世界的接触：磨损的机器人、实验室的时间、盯着的人、那次你没法撤销的撞车。

现实向你收取的是经验。模型让你把一部分账用计算时间来付。

你可以问「如果」。如果我现在刹车会怎样，如果我改成转弯会怎样，如果我从另一边推会怎样。能在不真的执行的前提下比较这些，正是「模型知道你指的是哪个动作」这件事之所以重要的全部理由。它把问题从*接下来大概会发生什么*，变成了*如果我做这件事会发生什么*。

要说准你得到的是什么。这个答案在模型内部是真的：以它学到的东西为准，早一点刹车会是这个结果。它不是一份来自那个并未发生的世界的报告。一摊没人看见的油、一阵风、任何模型手里从来没有过的东西，都会让这个答案自信地出错。

世界永远只会执行你真正做出的那个动作。模型是那些别的可能性得以存在的地方。

它在哪里出问题

到这里为止，所有内容都在支持「多来一点」：更好的模型、更多的候选、更长的推演，挑出赢家。这里每一步单看都很合理，而它们合在一起，会把你带进整个领域都绕着它转的那个失败。

有两件事会出问题，而它们不是同一件事。

误差会复利。一个只错一次、错一点点的模型没问题。一个错一点点、然后又去读自己那个答案的模型，会有一阵子还行，然后就差得没边了。没有哪一步是坏掉的。一个单步模型可以准到值得信任，而用同一个模型缝出来的一百步幻想却一文不值。所以

「往前看多远才划算」是有上限的，而最后一节里那个办法之所以那么粗暴，原因就在这里。

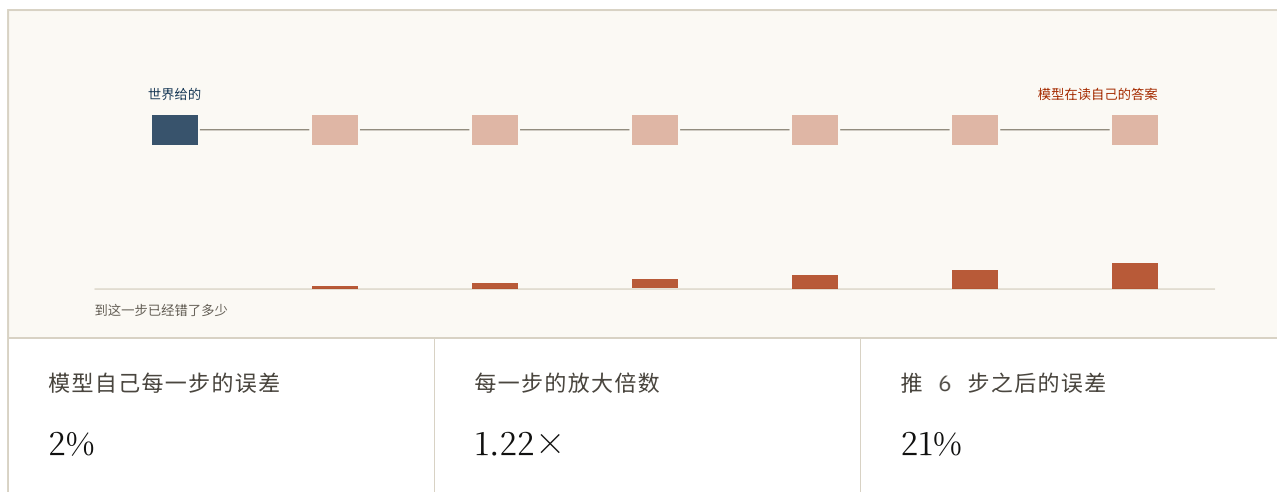


FIG. 2.5 每一步同时发生两件事：模型引入 2% 属于它自己的误差，而已经错掉的部分会被它所穿过的动力学再拉大一截。把放大倍数拖到 1.00，第二件事就关掉了：误差只是单纯累加，柱子排成一条直线。让它大于 1，柱子就开始往上弯。这两个数字都是示意性的；形状不是。

搜索会主动去找那些错误。这一件更古怪，也是这一章存在的理由。

假设学出来的地图只在一个地方错了：它以为墙上有个缺口，而那面墙其实是实心的。随机去试方案，你几乎永远不会靠近那里。弱一点的搜索永远找不到它。彻底一点的搜索会找到，因为在整个「学出来的世界」里，最好的路线正好从那儿穿过去。

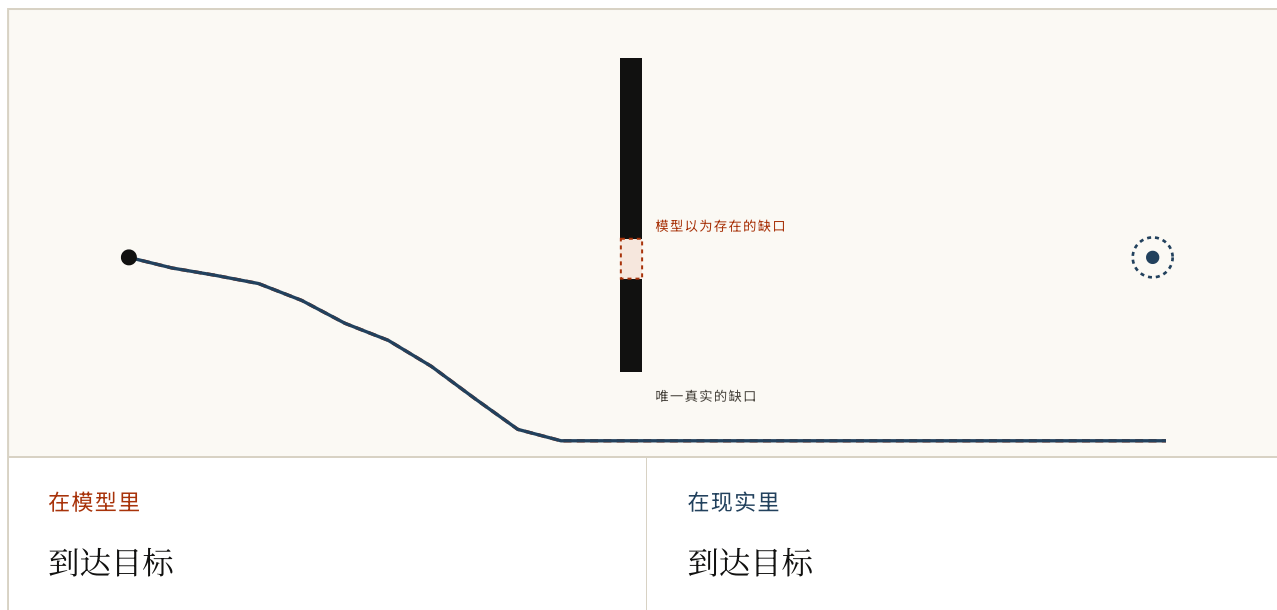


FIG. 2.6 模型相信墙上有个缺口。并没有。把力度留在低位，规划器根本找不到它，于是老老实实绕远路，然后到达了目标。接着把力度往上调。这是这套设定让你看见的一种失败方式，而不是「搜索越多结果一定越差」的定律。

这一幅要看两遍，因为它跟大多数工程直觉是反着来的。较弱的搜索给出了一个能用的方案。更强的那个给出了一个分数更高、然后撞墙的方案。它并没有跟你作对。它做的正是你要求它做的事：找出模型最喜欢的那个方案。

这个领域把它叫做**模型利用**。规划器越擅长在模型上拿高分，它就越会往模型从来没被教过的那些地方漂，而那些地方恰恰是它出错的地方。那些从一堆固定的、录好的经验里学习的系统 (叫做离线：没有办法再出去，把你后来发现自己需要的那份数据收回来) 受影响最重，因为它们没法去拿到那些能纠正自己的经验。

所以一个平均起来几乎不出错的模型，仍然可能是个危险的、不该在里面做规划的东西。平均误差问的是：在你度量过的那些情况上它表现如何。控制问的是：一旦有东西开始有目的地去搜刮它能找到的最高分，会发生什么。这不是同一个测试。

模型里最好的方案和世界里最好的方案，只有在模型于*搜索会去的地方*也够准时，才会是同一个方案。下一节里的所有内容，都是在设法让最后这半句成立。

$\arg \max_a$ 模型给出的分数 $\neq$ $\arg \max_a$ 实际发生的结果

除非模型在搜索会去的地方也足够准。

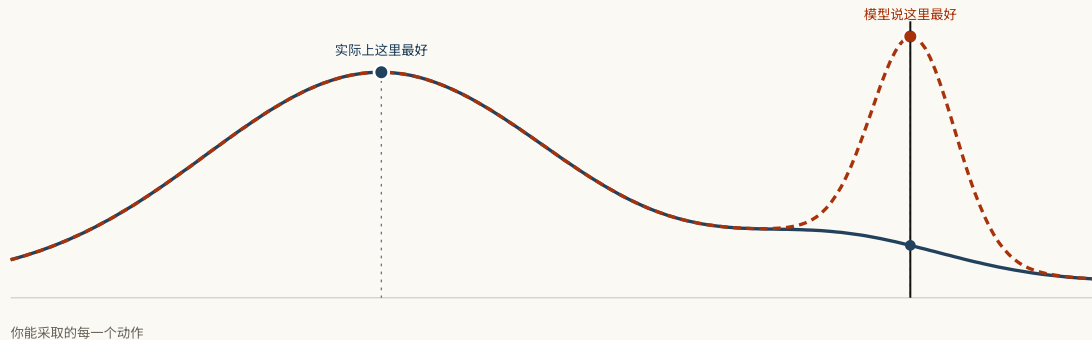


FIG. 2.7 给同一批动作打分的两种方式。它们几乎处处一致，而这正是「平均误差很低」看起来的样子。拖到那个尖峰上，把两个数字都读一遍：模型最中意的那个动作，在想象里得 0.97 分，在真实世界里得 0.20 分。

规划，就是把一个模型作为输入，产出或改进一个用于和被建模的那个世界打交道的策略。

Sutton 谈规划就是在一个学出来的模型上做计算，出自把整个问题立起来的那篇论文 (Dyna, an Integrated Architecture for Learning, Planning and Reacting, 1991)

<https://mlanthology.org/icml/1990/sutton1990icml-integrated/>

说得非常准，而困难就坐在被建模的那个世界这几个字里。你拿到的是一个对模型里那个世界很好用的策略。那是不是你正站着的这个世界，是另一个问题，而模型内部没有任何东西能回答它。

大家拿它怎么办

没有人一次性解决了这件事然后就翻篇了。这个领域攒下来的，是一套限制「想象能被信任到多远」的办法，而它们其实是同一条指令换了不同的衣服。

多问几个模型。 训练好几个而不是一个 (集成：同样的数据，不同的起点，于是它们在数据从未定下来的地方最终会互相矛盾)，然后把它们之间的分歧一路带进方案里。它们意见不一致的地方，就是方案漂到了没有任何东西支撑的地方。

分歧是个提示，不是判决。在同样的数据空白上训练出来的模型，可能共享同一个盲点，然后自信地在一件根本不存在的事情上达成一致。而模型自己的置信度也好不到哪去：当它说自己有七成把握时，它有七成的时候是对的，前提是真有人去核对过。有人去核对了，发现它经常不准，还发现把这个置信度校正过来之后，规划变好了。

别做太远的梦。 有一个办法简单到近乎粗鲁：让每一次想象出来的推演都从世界真正产生过的某个地方出发，并且保持它很短。你失去了模拟那种免费的无限。你换来的是「一个误差在真实数据打断它之前，能长多久」的上限。

按短时钟做规划也是同一件事：规划一点，执行一点，再看一眼，再规划 (这件事的标准名字叫模型预测控制，机器人学里大半都是它)。重规划没法去掉模型到处都在犯的那种系统性错误，但它确实在不停地把机会交还给世界，让世界来说东西究竟在哪儿。

为「不知道」收费。 把模型交回来的分数拿过来，在模型没把握的地方扣掉一块。这样一来，一条看着面生的捷径就必须好到足以抵消「没人知道那底下有什么」的代价。

这三条底下是同一句话：*不要让规划器因为跑去了一个模型还没资格自信的地方，而拿到满分。*

这些都没让问题消失。这些系统里最好的那些，如今能在惊人广的一批任务上站得住。但它们是使用一个错误的模型时足够小心的办法。它们不是「有一个正确的模型」的证明。

问题从来都不是怎么把模型做准。问题是：在哪里准，对哪些动作准，往前多远还准，面对多强的搜索还准，以及当它不知道的时候，这东西该怎么办。

试一试

1 在刹车演示里，速度低于调校值时，固定触发的那辆车和带模型的那辆车行为完全一样。这说明了什么？

- a. 动力学模型什么也没干
- b. 在一条缓存规则当初被准备好的条件范围内，它完全够用
- c. 基于模型的控制只在高速时才有意义
- d. 无模型的系统用不了速度信息

答案 b. 在一条缓存规则当初被准备好的条件范围内，它完全够用. 重点不是模型到处都赢。在缓存下来的答案仍然正确的那个范围里，重算一遍除了重算的开销之外什么也换不来。

2 一个循环策略把观测和记忆直接映射成动作，但它内部没有一个可以在「没执行过的动作」下往前推的转移函数。它是基于模型的吗？

- a. 是，因为它有记忆
- b. 是，因为任何大网络内部都含着一个模型
- c. 不是
- d. 只有当它的策略是随机的时候才算

答案 c. 不是. 记忆和复杂度本身并不构成一个可以调用的动力学模型。缺的那份契约，是在假设性动作下预测后果的能力。

3 某个系统把一帧摄像头画面编码成 512 个数，在这些数上把 500 条候选动作序列各推一遍，然后挑出最好的。它从不解码出未来的图像。它算数吗？

- a. 不算，因为世界模型必须预测像素
- b. 算，因为它学出来的状态可以在动作条件下往前推演
- c. 只有当这 512 个数对应到有名字的物理量时才算
- d. 只有当预测是确定性的时候才算

答案 b. 算，因为它学出来的状态可以在动作条件下往前推演. PlaNet、MuZero 和 TD-MPC2 正是「有用的世界模型必须在视觉上复现未来」这个想法的反例。与决策相关的潜在动力学就够了。

4 Dreamer 用它的世界模型生成的轨迹训练一个 actor，然后由 actor 直接选动作。那一刻并没有大规模搜索在跑。世界模型就不算数了吗？

- a. 不算了，因为规划必须发生在推理时
- b. 还算，因为动力学仍然生成了动作条件下的想象经验
- c. 不算了，除非 actor 把像素重建出来
- d. 这只取决于 actor 有多大

答案 b. 还算，因为动力学仍然生成了动作条件下的想象经验. 在线搜索只是一个可推演动力学模型的用法之一，而不是它的定义。Dreamer 把模型的开销花在更早的训练阶段，而不是动手的那一刻。

5 在模型利用那幅图里，把搜索预算调高之后规划器反而变差了。发生了什么？

- a. 优化器变得不准了
- b. 世界变难了
- c. 更好的优化找到了弱搜索错过的那个模型错误
- d. 长方案总是比短方案差

答案 c. 更好的优化找到了弱搜索错过的那个模型错误. 优化器只是更擅长把模型给出的分数拉高了。错的是那个假设：这个分数在搜索能够到的每一个地方，都还忠实于现实。

6 为什么短模型推演会有帮助？

- a. 神经网络做不了几步以上的预测
- b. 它限制了模型误差和分布偏移在真实数据回来之前能累积多久
- c. 短视野让模型变精确
- d. 它消除了不确定性

答案 b. 它限制了模型误差和分布偏移在真实数据回来之前能累积多久. 这正是 MBPO 那种「从真实状态出发的短推演」的动机：用模型用到足以拿到合成经验，但别用到让累积偏差压倒一切。

7 集成里的五个模型都认为某条捷径是安全的。这证明了什么？

- a. 这条捷径是安全的
- b. 失败的概率为零
- c. 只证明了这五个意见一致；共享的盲点仍然可能存在
- d. 证明了不需要更多模型

答案 c. 只证明了这五个意见一致；共享的盲点仍然可能存在. 分歧是有用的不确定性信号，PETS 用的就是它。但学出来的不确定性本身可能没校准好，在同样有偏的数据上训练出来的模型也可能一起自信地出错。

8 你的动力学模型说：如果你早一秒刹车，你会在撞墙之前停住。你手里拿到的到底是什么？

- a. 物理世界真正会怎么做的证明
- b. 在一个假设性动作下、相对于模型而言的预测
- c. 那条平行历史的录像
- d. 一个与隐藏变量无关的因果结论

答案 b. 在一个假设性动作下、相对于模型而言的预测. 这就是基于模型的控制里「如果」有用的那层意思。它让你在动手之前比较不同动作，而它的权威只延伸到那个学出来的模型所及之处。

FIG. 2.8 八道题，考的是这里真正重要的那个区分。不是「这个智能体看起来聪不聪明」，而是它在哪里算出后果，以及这份计算什么时候被允许跑到证据前面去。

上面这一切都悄悄假设了：你手里已经有一个可以往前推的状态。一个位置，一个速度，一小串装着「未来取决于什么」的数字。没有人会把它交给你。你拿到的是像素。

多谢阅读。第 3 章讲的是：*预测下一个东西*，这个听起来简单得根本不该管用的目标，为什么足以把一个世界的结构从原始经验里拽出来。

参考资料

World Models HA & SCHMIDHUBER, 2018 ↗

编码器、潜在动力学、紧凑控制器，还包括把策略完全放在模型生成的环境里训练、再搬回真实环境的实验。

<https://arxiv.org/abs/1803.10122>

Learning Latent Dynamics for Planning from Pixels (PlaNet)

HAFNER ET AL., 2018 ↗

从图像里学出随机潜在动力学，然后在决策时对它做搜索。图 2.4 的真实版本。

<https://arxiv.org/abs/1811.04551>

Dream to Control (Dreamer) HAFNER ET AL., 2019 ↗

在想象出来的潜在轨迹上训练 actor 和 critic，于是动手的那一刻不必再跑昂贵的搜索。

<https://arxiv.org/abs/1912.01603>

Mastering diverse control tasks through world models (DreamerV3)

HAFNER ET AL., NATURE 2025 ↗

同一个算法、同一套超参数，跑了 150 多个任务。这是想象这一支目前最强的证据。

<https://www.nature.com/articles/s41586-025-08744-2>

Mastering Atari, Go, chess and shogi by planning with a learned model (MuZero)

SCHRITTWIESER ET AL., 2020 ↗

最清楚的一个证明：规划并不需要重建观测。模型只学搜索会消费的那些量。

<https://arxiv.org/abs/1911.08265>

TD-MPC2: Scalable, Robust World Models for Continuous Control

HANSEN, SU & WANG, 2024 ↗

没有解码器的潜在动力学，配上局部轨迹优化，扩展到覆盖 104 个控制任务的单个多任务智能体。

<https://arxiv.org/abs/2310.16828>

Deep RL in a Handful of Trials using Probabilistic Dynamics Models (PETS)

CHUA ET AL., 2018 ↗

集成加轨迹采样：把不确定性做进规划里，而不是当作事后补丁的标准做法。

<https://arxiv.org/abs/1805.12114>

Dyna: an Integrated Architecture for Learning, Planning and Reacting

SUTTON, 1991 ↗

把规划定义成在学出来的模型上做计算，以及那个把它和真实经验交织起来的循环。

<https://mlanthology.org/icml/1990/sutton1990icml-integrated/>

Recurrent world models for planning and curiosity SCHMIDHUBER, 1990 ↗

把控制器和世界模型当作两个分开的对象，也就是这一章开头讲的那个区分。

<https://people.idsia.ch/~juergen/world-models-planning-curiosity-fki-1990.html>

Planning with an Adaptive World Model THRUN, MÖLLER & LINDEN, 1990 ↗

通过交互学出一个世界模型，再把它串起来优化未来的动作，比这个标签流行起来早了二十八年。

https://papers.nips.cc/paper_files/paper/1990/hash/9be40cee5b0eee1462c82c6964087ff9-Abstract.html

When to Trust Your Model: Model-Based Policy Optimization

JANNER ET AL., 2019 ↗

从真实状态出发的短推演，直接回应累积误差和模型利用。标题就是这一章的问题。

<https://arxiv.org/abs/1906.08253>

Benchmarking Model-Based Reinforcement Learning WANG ET AL., 2019 ↗

基于模型的方法到底在哪里赢、在哪里输，以及「往前看多远」这个两难在哪里被命名和度量。

<https://arxiv.org/abs/1907.02057>

Calibrated Model-Based Deep Reinforcement Learning MALIK ET AL., 2019 ↗

每一种不确定性方法底下的那句警告：不确定性估计本身可能就是错的，而校正它会改变规划的结果。

<https://arxiv.org/abs/1906.08312>

MOPO: Model-based Offline Policy Optimization YU ET AL., 2020 ↗

把悲观写进目标：按模型的不确定性给预测回报打折，于是面生的捷径必须为「面生」付出代价。

<https://arxiv.org/abs/2005.13239>

Quantifying the nature of anticipation in professional tennis

TRIOLET ET AL., 2013 ↗

比赛分析，把「最早可能是对球做出的反应」定在击球后大约 140 到 160 毫秒。需付费。

<https://doi.org/10.1080/02640414.2012.759658>

The spatiotemporal control of expert tennis players when returning first serves

NAVIA ET AL., 2021 ↗

开头那个 177 毫秒的出处，是测出来的，不是估出来的。需付费。

<https://doi.org/10.1080/02640414.2021.1976484>

FIG. 2.9 排序是给要在这上面接着做事的人用的，而不是为了历史完整性。